

PROGRAMMING FUNDAMENTALS

Spring 2024

24-04-2024

Array as a function Argument

- To pass an array as an argument to a function, pass the name of the array
- Functions could be written to :
 - put values in an array
 - display an array's contents on the screen
 - total all of an array's elements
 - calculate their average
- Such functions accept an array as an argument.

Passing a single array element

- When a single element of an array is passed to a function, it is handled like any other variable

Ex-1

`void showValue(int);` // Function prototype

```
int main()
{
    const int SIZE = 8;
    int numbers[SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};

    for (int index = 0; index < SIZE; index++)
        showValue(numbers[index]);
    return 0;
}
```

Ex-1

```
void showValue(int num)
{
    cout << num << " ";
}
```

Program Output

```
5  10  15  20  25  30  35  40
```

- Each time `showValue` is called, a copy of an array element is passed into the parameter variable `num`
- The `showValue` function simply displays the contents of `num` and doesn't work directly with the array element itself

Parameters vs Arguments

- **Parameters** are special-purpose variables inside the parentheses of a function **definition**
 - Arguments are listed inside the parentheses of a function call
 - The values in the parameter variables are copies of the arguments
- if a parameter's value changes inside a function → no effect on the original argument

Example-2

```
#include <iostream>
using namespace std;

// Function Prototype
void My_Func(int myValue);
int main()
{
    int number = 100;
    cout << "number is " << number << endl;

    My_Func(number);

    // Display the value in main() function
    cout << "Now back in main again, the value of ";
    cout << "number is " << number << endl;
    return 0;
}

void My_Func(int myValue)
{
    // Change the value of myValue
    myValue = -myValue;

    // Display the value in myValue.
    cout << "Now the value is " << myValue << " in my Function"<<endl;
}
```


Output

 D:\Books-ICT\C-Programs\Lec17-Code.exe

```
number is 100  
Now the value is -100 in my Function  
Now back in main again, the value of number is 100  
  
-----  
Process exited after 0.113 seconds with return value 0  
Press any key to continue . . .
```

Pass By Value

- My_Func() function does not have access to the original argument
- When only a copy of an argument is passed to a function, it is said to be *passed by value*
- The called function receives a copy of the argument's value and does not have access to the original argument
- *One disadvantage of pass-by-value is that, if a large data item is being passed, copying that data can take a considerable amount of execution time and memory space.*

Example-3

- Write a function ShowMenu() to display the following:

 D:\Books-ICT\C-Programs\Lec17-Ex3.exe

```
Library Membership Menu  
  
1. Standard Membership  
2. Child Membership  
3. Senior Citizen Membership  
4. Quit the Program  
  
Enter your choice:
```

Example-3

- Write a main function that calls the ShowMenu() function to display the main menu
- Declare a choice variable (int) in main() that will get the user's choice
- main() function also asks the user to enter the number of months for which membership is required

Example-3

- The constant membership rates per month are given below:

Rate for Standard membership = Rs. 400

Rate for Child membership = Rs. 200

Rate for Senior Citizen membership = Rs. 300

These need to be defined in main() function

Example-3

- Write three functions (`calc_fee1()`, `calc_fee2()`, `calc_fee3()`) which return the membership fee after calculating according to the constant rates defined.
- These functions will need the rate and the number of months to calculate the fee
- The returned fee is displayed in `main()` function
- Use switch case structure to call the appropriate function with required parameter values
- **Validation :** make sure that the user enters only valid choice

Example-3

- The main() function will call the appropriate function based on user's choice to calculate the membership charges.

Sample Run

 D:\Books-ICT\C-Programs\Lec17-Ex3.exe

Library Membership Menu

1. Standard Membership
2. Child Membership
3. Senior Citizen Membership
4. Quit the Program

Enter your choice: 1

Enter the number of months for which membership is required : 10

YourMembership Fee = 4000

Process exited after 6.008 seconds with return value 0

Press any key to continue . . .

Pass by Reference

- You place an ampersand (&) in front of the variable name
- **Passing a variable by reference** → "passing its address to the function"
- Only creates a pointer to the memory
- Function can directly access data in memory
- Changes affect original arguments' value
- *Pass-by-reference can weaken security; the called function can corrupt the caller's data.*

Reference Parameter

- Use & after data type in the prototype
- For Example:

```
int my_func1 (int& num);
```

We should read as : num is a reference to an integer

Ex-4

- Write a user defined function that calculates square of a number entered by the user.
- Use Pass By Value

```
int square_byVal(int num)
{
    num = num * num;
    return num;
}
```

Writing same function

- Using Pass by Reference

```
int square_byRef(int &num)
{
    num = num * num;
    return num;
}
```

Lets write the main() function

- To call the first function (pass by value)

```
int main()
{
    int n;
    cout<<"Enter a number to find square = ";
    cin>>n;
    int answer = 0;
    answer = square_byVal(n);
    cout<<endl<<"Returning to main() "<<endl;

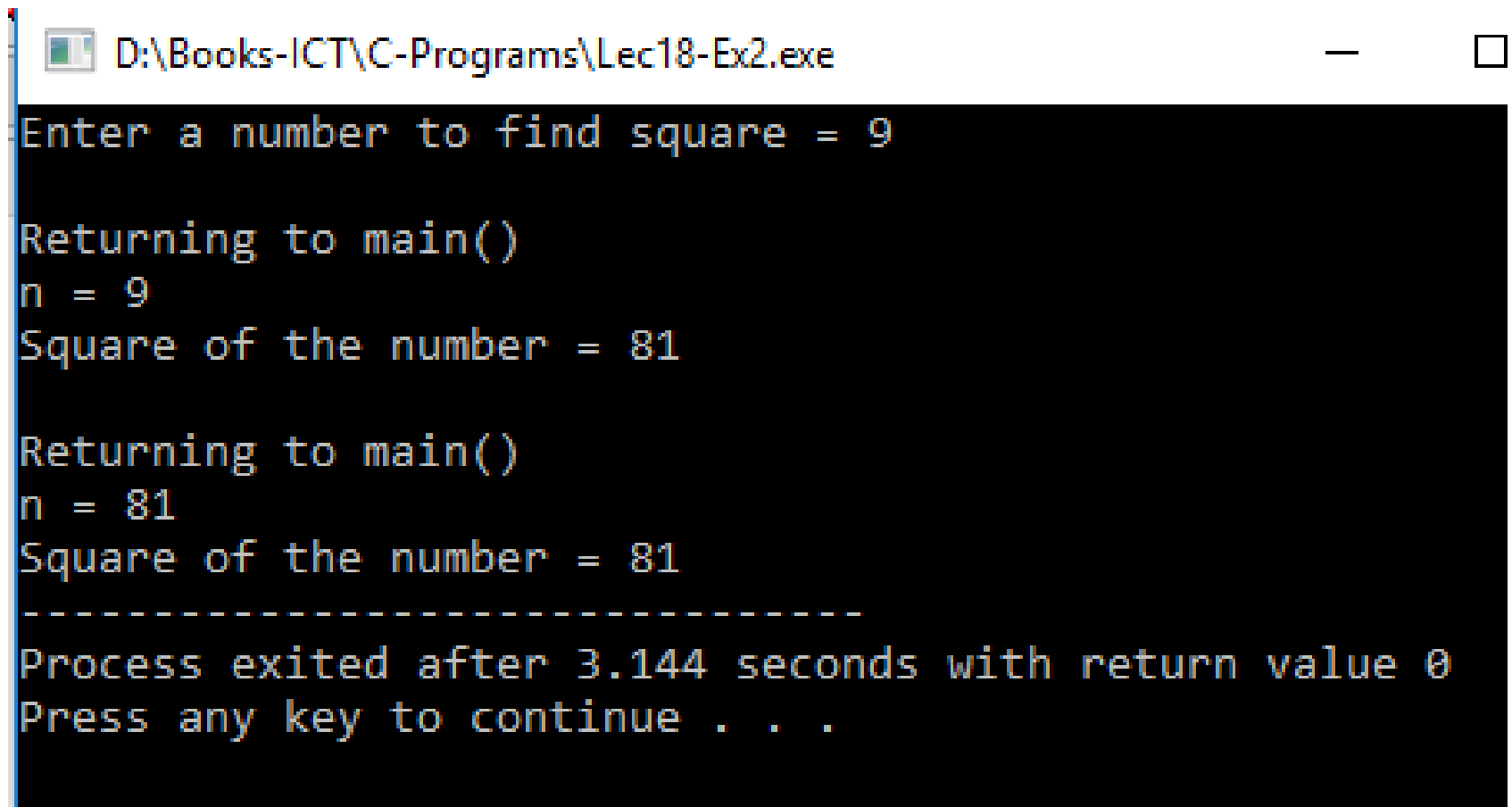
    cout << "n = "<<n<<endl;
    cout<< "Square of the number = "<<answer;
```

Modifying main()

- To call function with pass by reference strategy

```
int ans = 0;  
ans = square_byRef(n);  
cout<<endl<<endl<<"Returning to main() "<<endl;  
  
cout << "n = "<<n<<endl;  
cout<< "Square of the number = "<<ans;
```

Sample Run..



```
D:\Books-ICT\C-Programs\Lec18-Ex2.exe

Enter a number to find square = 9

Returning to main()
n = 9
Square of the number = 81

Returning to main()
n = 81
Square of the number = 81
-----
Process exited after 3.144 seconds with return value 0
Press any key to continue . . .
```

Ex-5

- Lets write a simple user defined function (void) which is called with two integers from main()
- The function swaps the two integers and displays the first and second number

The main() function

```
int main()
{
    int i = 10, j = 20;
    swapByVal(i, j);
    cout<<endl<<"Returning to main() "<<endl;
    cout << "i = "<<i << " " << "j = "<< j << endl;
    return 0;
}
```

The function prototypes

}-Ex1.cpp

Lec18-Ex2.cpp

```
#include <iostream>
using namespace std;
void swapByVal(int num1, int num2);
void swapByRef(int& num1, int& num2);

int main()
{
    ...
}
```

The user defined function

- Using Pass by Value

```
void swapByVal(int num1, int num2) {  
    int temp = num1;  
    num1 = num2;  
    num2 = temp;  
    cout<<"Function for PASS BY VALUE"<<endl;  
    cout<<"first no. = "<<num1<<endl;  
    cout<<"second no. = "<<num2<<endl;  
}
```

Write the function

- Using Pass by Reference

Function

```
void swapByRef(int& num1, int& num2) {  
    int temp = num1;  
    num1 = num2;  
    num2 = temp;  
    cout<<endl<<"Function for PASS BY REFERENCE"<<endl;  
    cout<<"first no. = "<<num1<<endl;  
    cout<<"second no. = "<<num2<<endl;  
}
```

Sample Run..

 D:\Books-ICT\C-Programs\Lec18-Ex1.exe

```
Function for PASS BY VALUE
```

```
first no. = 20
```

```
second no. = 10
```

```
Returning to main()
```

```
i = 10  j = 20
```

```
Function for PASS BY REFERENCE
```

```
first no. = 20
```

```
second no. = 10
```

```
Returning to main()
```

```
i = 20  j = 10
```

```
-----  
Process exited after 0.1475 seconds with return value 0
```

```
Press any key to continue . . .
```

Output explained..

- The addresses of `i` and `j` are passed
- The compiler knows they are references so `i` and `j` of main's memory are changed
- If you aren't going to change a variable, you use pass by value

Ex-6 Function to accept an entire Array

```
void showValues(int nums[], int size)
{
    for (int i = 0; i < size; i++)
        cout << nums[i] << " ";
}
```

- nums is not actually an array
- It's a special variable that can accept the address of an array
- When an entire array is passed to a function, it is not passed by value, but passed by reference
- only the starting memory address of the array is passed

Function Prototype

`void showValues(int nums[], int); // Function prototype`
or

`void showValues(int [], int); // Function prototype`

- In the function prototype, empty brackets appear after the data type of the array parameter
- This indicates that `showValues` accepts the address of an array of integers.

Main () Function

```
int main()
{
    const int ARRAY_SIZE = 8;
    int numbers[ARRAY_SIZE] = {5, 10, 15, 20, 25, 30, 35, 40};
    showValues(numbers, ARRAY_SIZE);
    return 0;
}
```

in C++ the name of an array without brackets and a subscript is actually the beginning address of the array

Ex-7

```
int main()
{
    const int SIZE1 = 8; // Size of set1 array
    const int SIZE2 = 5; // Size of set2 array
    int set1[SIZE1] = {5, 10, 15, 20, 25, 30, 35, 40};
    int set2[SIZE2] = {2, 4, 6, 8, 10};

    // Pass set1 to showValues.
    showValues(set1, SIZE1);

    // Pass set2 to showValues.
    showValues(set2, SIZE2);
    return 0;
}
```

Function

```
void showValues(int nums[], int size)
{
    for (int i = 0; i < size; i++)
        cout << nums[i] << " ";
}
```

Output

```
5  10  15  20  25  30  35  40
2  4   6   8  10
```

Book Reference

- Tony Gaddis
- Ch-6